多智能体协同开发系统搭建方案

Orchestrator Windows 主环境试点版 3.2 / 2026 年 10 月 12 日

决策摘要

首期使用 Windows AO 原生编排；DSH 的天翼 ACP 与 AO Chat 模型探测通过，Kimi API、CLI、原生 ACP 通过但 AO Chat 认证预检仍失败。Codex GPT-6.1 Sol 的 AO Chat 调用已通过，项目 orchestrator 模型已保存；Codex 只读权限与 Kimi TUI 结果仍未验收。权限和恢复验收前不触碰真实原型项目。

这些试验只证明部分会话启动、跨 Harness 派发和 worktree 隔离能力，没有证明目标 Harness 的认证、工具权限和审计已端到端接通。Claude 和 OpenCode 是验证用 Harness，不替代首期目标 Harness。AO 目录将 DeepSeek 标记为“已安装/已授权”，此前 ACP 仍指向 DeepSeek 官方 Provider；修正为天翼 ACP 专用模型路由后，直连与 AO Chat 最小请求均已通过。Kimi Code CLI 2.1.1 已安装；GPT 走官方订阅，其他模型走天翼 API。Kimi 已登记天翼 OpenAI-compatible 地址与 kimi-k3，用户级环境变量 `TIANYI_API_KEY` 已配置，HTTP API 与 Kimi CLI 探测通过；HTTP API、CLI 与原生 ACP 已通过，但 AO Chat 仍被 CHAT_AUTH_REQUIRED 阻断。ZCode 当前没有 AO 内置适配器，按用户决定暂不纳入首期。

决策项	MVP 决定
执行底座	仅 Windows Orchestrator。停止新建 WSL AO、Linux worker 和跨 OS worktree 映射；历史 WSL 环境保留，不清理、不重置。
任务流程	AO 原生 orchestrator 负责拆解、派发、跟踪与回收；复杂任务可采用 spec/plan/build/test/review/ship 检查清单，不强制每个任务走固定六次审批。
阶段管理	采用 AO 原生 orchestrator/worker/reviewer 编排，并在重要交付点由人确认；流程按任务复杂度裁剪，不开发独立状态机。AO 的会话状态和 PR/CI 状态用于跟踪，不宣称它们是自定义阶段的硬拦截器。
派发	由 AO 原生 orchestrator 创建隔离 worker 并回收结果；不另装 CCTeam/Omnigent，不在 AO 外再建派发控制面。
目标 Harness	Codex 负责规划、架构和终审，目标为只读；DeepSeek Harness 承担编码，Kimi Code 负责只读调查与复审。AO 项目权限没有显式 read-only 选项；Codex 只读边界需通过项目级 Codex sandbox 在 AO 会话中验证，未验证前不授予写任务。ZCode 暂不使用。
模型	GPT-6.1 Sol 通过 Codex 官方订阅使用；DeepSeek、Kimi、GLM 通过天翼 API 并共用 TIANYI_API_KEY。DeepSeek 请求 ID 为 deepseek-v4-pro-0813，接受并记录供应商实际返回标识，不自动切换；Kimi 使用 kimi-k3。GLM 暂不分配首期角色。
成本	不作为 MVP 目标；质量、可控性和完成率优先。保留 token 用量供诊断，不设降本比例。
发布	MVP 由人确认交付；不自动 push、合并或部署。

架构与任务流程

用户提出任务 → Windows AO orchestrator 规划与派发 → 按任务选择 Codex、DSH、Kimi 或其他已验证 Harness → AO 为 Git 任务建立独立 worktree → 使用 Harness 原生权限及已验证的角色 MCP → 测试/交叉评审 → 人工确认交付。复杂任务可用阶段清单；小任务可按风险裁剪。

spec → plan → build → test → review → ship 作为可裁剪的质量检查清单，不是固定工作流。AO orchestrator 根据任务复杂度决定需要哪些角色和步骤，重要交接由人确认。首期不单独实现阶段状态机；AO 会话状态和项目规则用于协作跟踪，不冒称不可绕过的程序级门禁。权限仍须逐 Harness 核验。

Orchestrator 的 Windows daemon、项目 orchestrator、worker/reviewer、Harness 适配、Git worktree 和会话 UI 是首期直接复用的协调层。AO 提供的 Chat MCP 能力取决于 Harness/Provider；MCP 网关不是 AO 的统一全局权限代理。每个 Harness 必须分别核验可见工具、原生文件/终端权限、取消和审计。AO daemon 保持本机默认访问边界，不另开公网入口。

Harness 与模型职责

Harness / 角色	职责	MVP 状态与限制
Codex + GPT-6.1 Sol	需求分析、架构、计划、只读终审	需求分析、架构、计划、只读终审。mh-pilot-11 的 AO Chat 以 gpt-6.1-sol 返回验证标记；项目 orchestrator 默认模型已保存为该型号。AO 权限枚举没有显式只读模式，Codex 只读边界仍未验收；旧 doctor 启动参数警告也未针对本次角色配置复验。
DeepSeek Harness	有明确验收条件的编码任务	DSH 0.1.2-alpha.5；天翼 ACP/AO Chat 通过；workspace-write 可越出任务文件范围，read-only 拒写/升级审批已验；kill 能停子进程并保留 dirty worktree，clean worktree 回收。restore/审计仍待验。
Kimi Code	只读代码调查、计划挑刺和独立复审	CLI 2.1.1；天翼 HTTP API 与 CLI 提示探针通过。AO Chat spawn 返回 CHAT_AUTH_REQUIRED；Kimi ACP 认证门只认 Kimi OAuth，未接受自定义 API Key；现有 TUI 会话结果未确认。
暂不纳入首期	暂不安排 Harness 职责	AO 当前无内置适配器；不为首期增加自定义桥接。以后需要时再评估官方 CLI/app-server 的权限和协议。
OpenCode	不纳入首期 Harness 角色	WSL 中历史完成的一次隔离任务仅作参考，不再继续 Linux 验证。

Harness 名称不等于模型身份。API 调用固定请求模型 ID；实际返回标识单独记录。供应商改路由时继续当前响应，不自动换模型。角色分工只是初始假设，须用相同任务集比较质量后才能扩权。

复用、自研与后续扩展

分类	组件	落地边界
复用开源	Windows Orchestrator、内置 Harness adapters、Git worktree、ACP、Playwright	AO 承担协调、任务 worker、review、会话生命周期和 worktree；DSH/Kimi 使用 AO 已提供的 ACP adapters；不自研 Harness 循环。

分类	组件	落地边界
参考但不部署	CCTeam、Omnigent/Polly、Vibe Kanban、Harnss	参考跨 Harness 会话/结果回收、任务看板、ACP/MCP 配置和派发前策略 hook。它们与 AO 的协调/隔离能力重叠；CCTeam 文档未说明 per-worker worktree 隔离且 Windows 路径要求 WSL，Harnss 公开标注早期开发且约 384 stars，不作为生产底座。
保留的现有代码	WSL 工作树中的 SQLite controller、MCP gateway、模型代理原型	本轮不启动、不读写、不移植、不删除。后续如需复用，先在隔离副本盘点可移植部分；不做 Windows↔WSL 桥接。
需自研/补齐	各 Harness 角色权限与审计验证；仅在用户批准后评估缺失的 adapter 或 MCP 代理	首期先用 AO 现有编排和 Harness 权限配置。发现具体能力缺口时先报告证据和影响，再由用户决定是否开发；外部状态机不在首期范围。
二期扩展	CAO 对照、更多 Harness、自动角色选择、自动 Draft PR、并行多写者	只有 M0/M1 权限和恢复验收通过后再评估；当前不启用自动发布或自动修复。

Orchestrator.inc 官方 Windows 发行版与本机实测最贴合当前目标；GitHub 仓库约 13.1k stars。其内置 adapter 矩阵包含 Codex、Kimi、DeepSeek Harness 等，但 Worker/Chat/Review 支持各异，按官方能力矩阵逐项验收。CCTeam（648 stars）证明 MCP 派发和策略 hook 可以驱动多 Harness；其公开文档未说明 per-worker worktree 隔离，且 Windows 使用路径要求 WSL，因此只借鉴派发/拒绝策略，不在首期部署。Omnigent（约 10.7k stars）与 AO 的多 Harness/worktree 能力重叠；Vibe Kanban（约 28.3k stars）偏任务/worktree 管理，未证明同一任务内跨 Harness 相互派发；Harnss（384 stars）虽有 ACP + per-project MCP 结构，但仓库明示早期开发/大改写，暂不部署。调研细节见《多 Harness 参考项目调研》。

权限、凭据与失败规则

- 规划和评审角色必须只读。AO 没有通用只读权限；Codex 的 AO Chat 只读边界仍未验。DSH 在 DSH_PERMISSION_MODE=read-only 下直接写入被拒，升级请求进入 AO pending 审批；worker permissions=default 已验证。workspace-write 仅限制 worktree，不限制任务文件范围。
- Codex 使用官方订阅身份；DeepSeek、Kimi、GLM 等天翼模型共用用户提供的同一 API Key，并通过 `TIANYI_API_KEY` 供 Harness 读取。天翼供应商不支持按角色撤销该 Key；当前 AO daemon 环境含该变量，子 Harness 进程可能继承（包括 Codex），per-Harness 环境隔离尚未验证。不得声称凭据已按角色隔离；不写入任务卡、提示或审计。首期按用户决定允许代码发往第三方云 API；审计不保存代码原文、完整提示或密钥。
- - MCP 网关负责其代理的文件读写、终端、Git、Playwright 工具。Harness 原生工具还须通过 Harness 权限配置与 Windows 进程/文件系统隔离；未证明能限制任务外读写的 Harness 不开放写权限。
- 每次 API 调用预期记录请求模型 ID、返回模型标识、token 用量、耗时、状态码、角色、Harness 版本、task_id 和错误类别。AO SQLite 会话活动详情实测保留 MCP 工具输入（路径和写入内容）；AO 原生记录不等于脱敏审计，当前尚未满足不保存代码原文的要求。
- 超时、限流和服务错误统一分类；单阶段最多额外重试 2 次。鉴权失败不重试；耗尽预算即失败阻断，不自动切模型、不自动修代码。
- 每个写任务用独立 worktree。失败保留现场；删除全量 checkout/reset/clean 回滚方案。清理或回滚仅由人工确认范围后执行。
- 浏览器使用任务独立临时用户目录；验收检查最终 URL、关键 DOM、控制台/页面错误、网络失败和 HTTP 错误，截图只作补充证据。about:blank、意外重定向或残留 Profile 锁均阻断。

- AO 会话历史会保存提示和部分工具输入；不得向会话输入密钥。要满足脱敏审计，需先核实 AO 保留策略，或评估单独的脱敏审计层。AO daemon 固定回环监听，禁止直接暴露远程端口。

M0 与后续里程碑

已完成的底座试验

项目	结果
Windows Orchestrator	daemon health/ready 通过；AO CLI 位于桌面安装包 resources\daemon\ao.exe，本机 ao 命令未在 PATH。Codex Chat 与 DSH Chat 有模型标记通过证据；Codex 只读权限尚未验收。
Windows DSH worker	天翼 ACP/AO Chat 模型探测通过。mh-pilot-13 证明 workspace-write 可越出任务文件范围；read-only 写入拒绝、AO 升级审批、kill 停止子进程与 dirty worktree 保留已验证；任务文件级权限、restore 与脱敏审计仍待测。
WSL2 OpenCode worker	历史试验在 Linux AO 的独立合成 Git 仓库完成单文件任务。新决策后不再作为运行或验证环境；保留历史记录，不继续操作。
首期 Harness 状态	Codex GPT-6.1 Sol AO Chat 通过但只读边界待验；DSH read-only 拒写、升级审批、取消与 dirty worktree 保留已验，restore/审计待验；Kimi 天翼 API/CLI 探针通过，但 AO Chat 被 Kimi ACP OAuth-only 检查阻断，TUI 未确认；ZCode 暂不纳入。
真实项目隔离	本轮只使用合成仓库；未向 Orchestrator 注册或修改真实原型项目。

M0 下一步

1. 保持真实原型项目未注册；继续使用 mh-pilot 核验目标 Harness、模型、权限与协议，并把 AO 目录授权和真实模型请求结果分别记录。
2. DSH read-only 直接拒写与 AO default 权限下的一次性升级审批已有证据；继续验证取消、失败恢复，并在切换 workspace-write 后核验编码任务能力。
3. GPT-6.1 Sol AO Chat 已通过；继续验证 AO 对 Codex 项目级 sandbox_mode=read-only 的强制效果。验证前 Codex 不承担写任务。
4. Kimi Code 已预配天翼 OpenAI-compatible provider 与 kimi-k3；Kimi 配置使用共享环境变量 TIANYI_API_KEY 后，处理 AO readiness unknown / CHAT_AUTH_REQUIRED，并确认 AO Chat 是否识别自定义 Tianyi Provider；若未解除，再单独验证 AO Terminal UI 路径。ZCode 暂不接入。
5. 不做 WSL 桥接。在 Windows AO 合成项目核验 orchestrator、worker/reviewer、各 Harness 的可用工具、原生权限、worktree 和人工确认流程。记录 AO 原生方式在实际任务中的表现；若出现影响交付的具体缺口，先提交证据和选项，再由用户决定是否补组件。

M0 的 Harness 认证、角色权限、worktree 隔离和失败恢复验收通过前不进入真实原型开发任务。已有 Linux 状态机/网关工作树保持原样，不启动、不读写、不清理、不重置。

M1 与 M2

- M1：在 Windows AO 合成仓库完成一条按任务需要裁剪的协作链：Codex 只读计划 → AO orchestrator 按任务派发 → 指定 Harness worker 实现 → 确定性测试 → 独立只读评审 → 人工确认交付。失败保留现场并验证人工恢复。
- **M2：对首期三个 Harness 逐一做真实角色任务和交叉评审，再在真实项目的只读快照/独立副本完成试点。ZCode 暂不纳入；未经用户授权不写回原始工作树、不 push、不合并、不部署。**

MVP 验收

指标	标准
目标 Harness 接入	Codex、DeepSeek Harness、Kimi Code 均以真实 CLI/API 完成各自角色任务；认证、模型标识、权限拒绝、取消和审计字段有证据。ZCode 因 AO 暂无 adapter 而排除在首期验收范围外。任一首期 Harness 未通过，不能宣称多 Harness MVP 闭环。
端到端交付成功率	预登记 10 个小中型任务，至少 9 个通过全部适用验收；人工补写代码单独统计，不算 Agent 自主交付成功。
人工干预	统计每任务计划内审批、计划外介入、人工补写三项；成功任务的计划外介入平均不高于 2 次。
缺陷逃逸	交付后 7 日内高严重度缺陷为 0；确认缺陷逃逸率不高于 10%，无确认缺陷时标记 N/A。
安全门禁	评审写入、Worker push、任务外写入、无授权数据库写均为 0；角色越权拒绝测试全部通过。
任务隔离与恢复	并行任务 worktree 不串写；失败现场可检查；重启/取消后不自动重放写操作。
浏览器	每任务 Profile 隔离；URL、DOM、控制台/页面错误和网络检查有证据。
成本	不设降本目标，也不设与 GPT 基线的成本门槛；token 只用于诊断。

稳定使用前累计观察至少 30 个任务。MVP 发布须附任务清单、版本与配置、角色权限矩阵、审计样例和未解决问题。

主要风险与依据

- 工具权限边界（高）：AO 提供的 MCP 能力依 Harness/Provider 而异，不能自动约束 Harness 原生文件、终端和 Git 工具。逐 Harness 验证权限；不能证明写入边界时，该 Harness 暂不执行写任务。
- **身份/模型误认（高）：** CLI 登录成功不证明请求模型可用；返回模型标识不证明供应商底层权重。模型映射和返回字段都应留证。
- **ZCode 暂不纳入首期：AO 当前没有内置适配器，不开发自定义桥接。若后续需要，再评估官方协议和权限边界。**
- Codex 只读边界（高）：AO 项目角色配置没有显式只读模式；把 Codex 指定为规划/评审角色不能证明其工具已被硬性限制为只读。必须在隔离 mh-pilot worktree 验证项目级 Codex sandbox，或在验证前禁用 Codex 写工具。
- Linux 原型可移植性（中）：原 WSL 状态机/网关依赖 Linux 系统调用与沙箱。首期不桥接、不操作，也不迁移。

Harness 就绪度不足（高）：DSH ACP 与 AO Chat 最小派发已通过；Kimi 的天翼 API、CLI、原生 ACP 已通过但 AO Chat 认证预检仍失败。不得把 AO 支持目录中的 adapter 等同于真实接入成功。

官方资料：

[1] Orchestrator.inc 项目与 worktree 说明：https://github.com/OrchestratorInc/agent-orchestrator

[2] Orchestrator Windows 安装与本地 Harness 认证：https://docs.orchestrator.inc/installation/

[3] Orchestrator OpenCode 原生 ACP 适配：
https://docs.orchestrator.inc/plugins/agents/opencode/

[4] Orchestrator 角色与 worker 委派：https://docs.orchestrator.inc/guides/per-role-agents/

[5] Orchestrator Harness 适配矩阵：https://docs.orchestrator.inc/plugins/agents/

[6] CAO 跨 Provider profile 参考：https://awslabs.github.io/cli-agent-orchestrator/docs/features/multi-provider/

[7] Orchestrator 支持的平台：https://docs.orchestrator.inc/platforms/

[8] Orchestrator DeepSeek Harness 适配说明：https://github.com/OrchestratorInc/agent-orchestrator/blob/main/docs/harnesses/deepseek-harness.md

[9] Orchestrator 当前 Harness/Chat 适配矩阵：https://docs.orchestrator.inc/plugins/agents/

[10] Agent Client Protocol 官方规范：https://github.com/agentclientprotocol/agent-client-protocol

[11] Kimi Code 官方 ACP 与 MCP 转发能力：https://www.kimi.com/code/docs/en/kimi-code-cli/reference/kimi-acp

[12] Harnss 仓库（早期开发状态、ACP engines、per-project MCP）：https://github.com/OpenSource03/harnss

2026-10-12 接入状态更新

- GPT-6.1 Sol 使用 Codex 官方订阅；DeepSeek、Kimi、GLM 通过天翼 API 并共用 TIANYI_API_KEY。密钥值没有写入文档或审计。
- DSH ACP profile 通过 apiKeyEnv 引用共享 Key；必须同时设置通用 agent-default-model 与 ACP 专属 acp route。直接 ACP 和 AO Chat 最小探测均通过，AO worktree 无改动、未提交。
- Kimi 天翼 HTTP API、CLI 2.1.1、原生 ACP 均通过；AO readiness 为 unknown，AO Chat 仍返回 CHAT_AUTH_REQUIRED。
- 本次 AO daemon 环境含 TIANYI_API_KEY，子 Harness 可能继承该变量，包括 Codex；per-Harness 环境隔离尚未验证，是当前高风险项。
- M0 仍需完成 Codex 只读权限验证、DSH 权限/取消/审计验证，以及 Kimi AO 接入路径验证；真实原型项目尚未触碰。

2026-10-12 Codex 角色模型与只读边界复核

- 截图旧脚本的 HTTP 探测返回 200，但 marker_match=False，不能记为 CLI 模型回复通过；红色 NativeCommandError 是 PowerShell 对原生命令 stderr 的处理。更新后的隐藏输出脚本复跑 Kimi Code CLI 2.1.1 成功，使用天翼共享 Key，未显示密钥或提示正文。
- mh-pilot 的 orchestrator 默认模型已保存为 gpt-6.1-sol，保留原 DSH worker 和其余可见配置。已有 mh-pilot-11 AO Chat 模型 smoke 返回验证标记；本轮创建新 orchestrator 会话的命令返回历史 mh-pilot-5，故项目角色新会话的模型生效仍待确认。
- AO set-config 的权限模式为 default、accept-edits、auto、bypass-permissions，没有显式 read-only。Codex CLI 支持 read-only sandbox，但 AO Chat 是否强制传递该限制没有实测；未通过前 Codex 不执行写任务，且没有修改全局 Codex 设置。
- Kimi AO session standalone-1 当前为 working，模型 tianyi/kimi-k3，最终回复未核验；Kimi AO Chat 仍有 CHAT_AUTH_REQUIRED。此 session 不记为通过。
- 天翼 DeepSeek、Kimi、GLM 共用同一 API Key；AO daemon 子 Harness 是否继承环境变量仍未验证。真实原型项目未触碰。

2026-10-12 DSH 隔离写入 smoke

- AO mh-pilot-12 使用 DSH Harness 在独立 worktree 执行了一个用户已明确授权的写入提示，生成 permission-probe.txt，内容为 DSH_WRITE_PROBE。worktree 只有该未跟踪文件，0 commit、未 push；合成仓库基线未改变。文件保留用于审查，没有自动回滚。

- AO 会话记录有 2 条已完成的 mcp_tool 活动，没有审批活动类别；安全筛选后的记录未能确认工具名称。因此这证明任务内写入可通过 AO MCP 活动完成及 worktree 隔离有效，不证明越权写入会被拒，也不证明存在覆盖 Harness 原生工具的独立权限网关。
- 下一步在合成 worktree 执行明确超出任务授权范围的拒绝探测，并核验取消与工具名、调用结果等审计字段。用户授权前不测试真实项目路径、数据库或远程 push。

2026-10-12 DSH 沙箱与 AO 审批复测

- mh-pilot-13 在默认 workspace-write 下通过 DSH write 工具创建任务外标记 OUT_OF_SCOPE_PROBE.txt；worktree 仅有该未跟踪文件，0 commit、0 push。提示词不能把工作区写权限缩小到任务文件范围。
- 合成项目 mh-pilot 现设 DSH_PERMISSION_MODE=read-only、worker permissions=default。mh-pilot-14 与 mh-pilot-16 的直接写入均被沙箱拒绝；mh-pilot-16 的一次性 workspace-write 升级进入 AO pending 审批，未批准，标记文件不存在，0 commit、0 push。
- AO SQLite conversation_activities.detail_json 持久化 MCP 工具输入，包括 file_path 与 content；当前不能宣称 AO 原生会话记录满足不保存代码原文的脱敏审计要求。
- 截图中的旧 Kimi CLI 探针因 marker_match=False 未通过；更新后的隐藏输出脚本已复测 CLI 成功。AO Chat 仍 CHAT_AUTH_REQUIRED，TUI 会话仍无可核验完成结果。真实原型项目未触碰。

2026-10-12 Kimi ACP API-Key 认证阻断

- 截图里的旧探针 HTTP 200 但 marker_match=False，不能算模型回答通过；更新后的隐藏输出 CLI 提示探针已返回标记并退出 0。没有显示 Key 或提示正文。
- Windows AO 对 Kimi Chat 的真实 spawn 返回 CHAT_AUTH_REQUIRED。AO agent catalog 显示 installed、authStatus=unknown；接口能力登记不等于会话认证成功。
- Kimi Code CLI 官方 ACP 代码的认证门检查 Kimi OAuth token；官方 PR #2185 记录了 API-Key 用户在 ACP 被强制 OAuth 登录的问题。该限制与 Tianyi API/CLI 探针成功、AO Chat 被拒绝相符。
- standalone-1 Kimi TUI smoke 仍为 working，最终回复未核验，不终止、不清理。mh-pilot-17 当前为 working/active，AO CLI 无可核验终端回复，worktree 只有基线文件；不计通过。需从 AO TUI 界面核验或等待上游修复 ACP API-Key auth gate。

2026-10-12 DSH 取消与 worktree 保留复测

- mh-pilot-18：45 秒无副作用的 Python sleep 子进程被 ao session kill 终止；因 worktree 干净，AO 自动回收目录，无任务改动丢失。
- mh-pilot-19：先创建单一未跟踪标记，再启动 sleep。kill 后 AO session 为 terminated，子进程归零；worktree 和标记仍存在，0 commit、0 push。标记留作验收证据，不清理。
- 已验证 kill 停止 DSH 子进程，并保留有未提交改动的 worktree；干净 worktree 会安全回收。session restore、复杂失败恢复和任务文件级写权限仍未验收。规则见 https://docs.orchestrator.inc/plugins/workspaces/worktree/。